

Name: Ambika

UID : 23BCC70026

Class : 23BCC-1(A)

Experiment – 6.2

Aim

To implement **protected routes** in a Node.js application using **JWT (JSON Web Token)** authentication, ensuring that only authorized users can access specific API endpoints.

Theory

♦ What is JWT?

JWT (JSON Web Token) is a secure, compact, and URL-safe way to represent user identity between client and server.

It is widely used for **authentication and authorization** in modern web applications.

A JWT consists of three parts:

1. **Header** – contains token type (JWT) and hashing algorithm (HS256).
2. **Payload** – contains user data (e.g., id, email, role).
3. **Signature** – ensures token integrity by verifying the sender's authenticity.

Format:

Header.Payload.Signature

◆ Protected Routes

Protected routes are API endpoints that can only be accessed by authenticated users who provide a valid JWT token.

Example Flow:

1. User logs in → Server generates a JWT token.
2. Client stores the token (usually in localStorage or cookies).
3. When accessing a protected route, the client sends the token in the **Authorization header**.
4. The server verifies the token using a secret key before granting access.

Procedure

1. Install dependencies (express, jsonwebtoken, dotenv, body-parser).
2. Create an Express server.
3. Implement a /login route that generates a JWT token when credentials are valid.
4. Create a **middleware** function verifyToken that:
 - a. Checks for token presence in headers.
 - b. Verifies it using jwt.verify().
5. Protect specific routes by applying the verifyToken middleware.
6. Test by making requests with and without a valid token.

Code Implementation

// File: server.js

```
const express = require("express");
const jwt = require("jsonwebtoken");
const bodyParser = require("body-parser");
require("dotenv").config();
```

```
const app = express();
```

```

app.use(bodyParser.json());

// Secret key for signing tokens
const SECRET_KEY = process.env.JWT_SECRET || "mysecretkey";

// LOGIN ROUTE (Token Generation)
app.post("/login", (req, res) => {
  const { username, password } = req.body;

  // Dummy authentication (replace with DB check)
  if (username === "admin" && password === "12345") {
    const user = { username }; // payload
    const token = jwt.sign(user, SECRET_KEY, { expiresIn: "1h" });
    return res.json({ message: "Login successful", token });
  } else {
    return res.status(401).json({ message: "Invalid credentials" });
  }
});

// MIDDLEWARE TO VERIFY TOKEN
function verifyToken(req, res, next) {
  const authHeader = req.headers["authorization"];
  const token = authHeader && authHeader.split(" ")[1]; // Bearer <token>

  if (!token) {
    return res.status(403).json({ message: "Token missing" });
  }

  jwt.verify(token, SECRET_KEY, (err, user) => {
    if (err) return res.status(401).json({ message: "Invalid token" });
    req.user = user; // attach user to request
    next();
  });
}

// ✅ PROTECTED ROUTE
app.get("/dashboard", verifyToken, (req, res) => {
  res.json({

```

```
    message: `Welcome ${req.user.username}!`,
    data: "This is protected content.",
  });
});

// ✅ PUBLIC ROUTE
app.get("/", (req, res) => {
  res.send("Public Route: Login to access protected routes.");
});

const PORT = 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Sample Output

1. Login Request

POST → /login

```
{
  "username": "admin",
  "password": "12345"
}
```

Response:

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLT..."
}
```

2. Access Protected Route

GET → /dashboard

Header:

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...

Response:

```
{  
  "message": "Welcome admin!",  
  "data": "This is protected content."  
}
```

3. Access Without Token

GET → /dashboard

Response:

```
{  
  "message": "Token missing"  
}
```

Learning Outcomes

By the end of this experiment, students will be able to:

1. Understand how **JWT authentication** works in Node.js.
2. Implement **secure login systems** using tokens.
3. Create **protected API routes** that verify JWT before granting access.
4. Use **middleware** to centralize token verification logic.
5. Understand the role of **authorization headers** in secure API communication.